

PROGRAMA DE GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Trabalho Prático 1

Desenvolvimento de um Sistema de Automação para Projetos em C (cbuild)

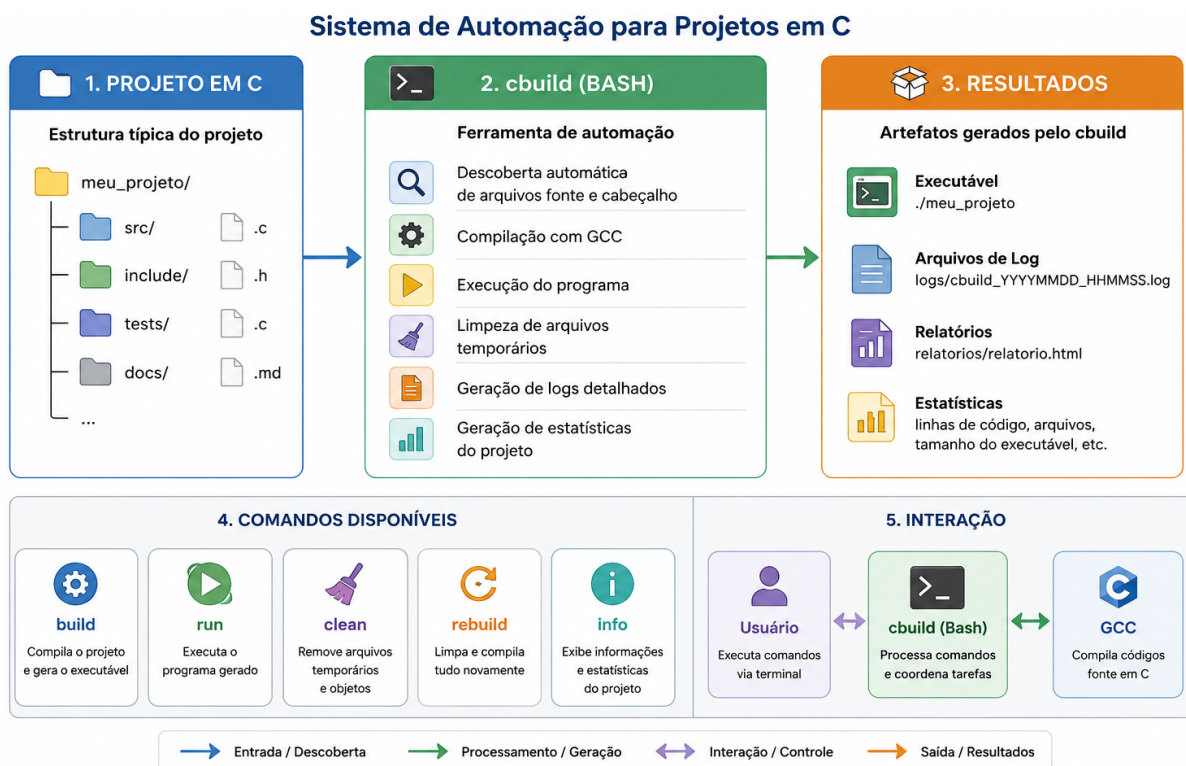


Figura 1. Arquitetura geral da ferramenta cbuild.

1. Objetivo

O objetivo deste trabalho é projetar, implementar e avaliar uma ferramenta de automação para projetos escritos em linguagem C utilizando Shell Script (Bash). A ferramenta deverá automatizar tarefas de compilação, execução, limpeza, geração de logs e obtenção de estatísticas do projeto.

2. Descrição Geral do Projeto

Os alunos deverão desenvolver uma ferramenta denominada cbuild, executada por meio da linha de comando. O sistema deverá operar sobre projetos em C organizados em múltiplos arquivos fonte e cabeçalhos, automatizando tarefas normalmente realizadas durante o desenvolvimento de software.

Especificação complementar: este enunciado define o comportamento mínimo esperado da ferramenta, mas não determina uma única estratégia de implementação. As decisões internas de projeto deverão ser tomadas pelo grupo e justificadas no relatório técnico. O objetivo é avaliar tanto o funcionamento quanto a capacidade de projetar e justificar uma solução em Bash.

3. Requisitos Funcionais Obrigatórios

3.1 Gerenciamento do Projeto

- Descobrir automaticamente arquivos .c e .h.
- Criar diretórios build/ e logs/ quando necessário.
- Identificar a estrutura do projeto.

3.2 Comandos

- build
- run
- clean
- rebuild
- info

3.3 Tratamento de Erros

- Diretório inexistente.
- GCC não instalado.
- Erros de compilação.
- Ausência de arquivos fonte.
- Permissões insuficientes.

3.4 Registro de Logs

Registrar data, horário, comando executado, tempo de execução, código de retorno e mensagens de erro.

3.5 Estatísticas

Exibir quantidade de arquivos, linhas de código, tamanho do executável e datas da última compilação e execução.

Quando uma informação ainda não existir, como a data de uma compilação que nunca ocorreu, a ferramenta deverá indicar explicitamente que o dado está indisponível.

O critério utilizado para contabilizar linhas de código deverá ser definido pelo grupo e aplicado de forma consistente. O relatório deverá informar se linhas vazias, comentários, diretivas de pré-processamento e outros casos particulares são ou não contabilizados.

Em uma execução bem-sucedida, o log deverá indicar claramente o sucesso da operação. Em uma execução com falha, deverá registrar a falha e, quando aplicável, informações suficientes para diagnóstico.

Cada registro deverá permitir distinguir operações diferentes. O grupo deverá definir e documentar o formato e o nome dos arquivos de log. Não será exigido um formato único, desde que todos os campos obrigatórios possam ser identificados.

Não é necessário adotar uma mensagem textual específica. A avaliação considerará se o erro é detectado, comunicado e registrado de forma adequada.

Em todos os casos de erro, a ferramenta deverá informar de maneira compreensível o que ocorreu e retornar código de saída diferente de zero. Além dos casos listados, deverão ser tratados comandos ou opções inválidas e a tentativa de executar o projeto sem um executável disponível.

Por padrão, considera-se que o comando é executado na raiz do projeto. Caso o grupo permita a indicação explícita de outro diretório, essa opção deverá ser documentada e funcionar de forma consistente nos comandos aplicáveis.

A descoberta dos arquivos fonte deverá ser realizada de forma recursiva a partir do diretório do projeto. Os diretórios build/ e logs/ são destinados aos artefatos gerados pela ferramenta e não deverão ser tratados como código-fonte.

4. Requisitos Técnicos

Implementação obrigatória em Bash utilizando funções, estruturas condicionais, laços de repetição, arrays, parâmetros da linha de comando, redirecionamento de entrada e saída e códigos de retorno. O projeto deverá executar em ambiente Linux.

5. Relatório Técnico

Apresentar arquitetura da solução, decisões de projeto, descrição da implementação, dificuldades encontradas, resultados obtidos e melhorias futuras.

O relatório deverá também descrever a interface de utilização, a estratégia de descoberta dos arquivos, a compilação incremental, o tratamento de erros, o formato dos logs, as estatísticas, as funcionalidades adicionais.

6. Demonstração

Cada grupo deverá demonstrar todos os comandos implementados, o tratamento de erros e o funcionamento da ferramenta.

A demonstração deverá incluir, além dos comandos normais, exemplos de falhas e das funcionalidades adicionais obrigatórias. O grupo deverá estar preparado para explicar e justificar as principais decisões de implementação.

7. Critérios de Avaliação

- Funcionamento da ferramenta: 30%
- Organização e modularização: 20%
- Tratamento de erros: 15%
- Uso adequado dos recursos do Bash: 15%
- Documentação: 10%
- Qualidade do código: 10%

8. Funcionalidades Adicionais (Obrigatório)

Compilação incremental; modo verboso; modo debug; escolha do nível de otimização; geração de relatórios; leitura de arquivo de configuração.

Arquivo de configuração: deverá permitir configurar pelo menos uma característica da ferramenta, como nome do executável, opções de compilação ou diretórios. O formato deverá ser documentado.

Geração de relatórios: deverá produzir um relatório sobre o projeto ou sobre suas operações. O formato fica a cargo do grupo e deverá ser documentado.

Escolha do nível de otimização: deverá ser possível selecionar pelo menos dois níveis de otimização, além da configuração padrão adotada pelo grupo.

Modo debug: deverá apresentar informações adicionais que auxiliem a acompanhar o fluxo interno da ferramenta e diagnosticar problemas.

Modo verboso: deverá apresentar informações adicionais sobre as etapas executadas e, quando pertinente, os comandos externos utilizados.

Compilação incremental: após uma compilação, arquivos que não sofreram alterações não deverão ser recompilados desnecessariamente. A estratégia utilizada deverá ser explicada.

Todas as funcionalidades desta seção são obrigatórias. O termo 'adicionais' significa que elas complementam os comandos básicos, e não que sejam opcionais.

9. Entrega

- Código fonte.
- Script executável.
- README.
- Relatório técnico em PDF. O relatório deverá conter introdução, arquitetura da solução, descrição dos módulos implementados, decisões de projeto, exemplos de execução, dificuldades encontradas, resultados obtidos e propostas de melhorias.
- Vídeo de demonstração de até 10 minutos.
- GIT – O repositório Git deverá apresentar histórico de commits compatível com a evolução do projeto, evidenciando a participação dos integrantes do grupo.

O histórico do Git deverá ser compatível com a evolução do projeto. Durante a apresentação, qualquer integrante poderá ser arguido sobre qualquer parte da implementação.

A avaliação individual poderá considerar a capacidade de explicar o funcionamento da solução e justificar as decisões tomadas pelo grupo.

O README deverá apresentar os pré-requisitos, a forma de execução, a sintaxe dos comandos, as opções disponíveis e exemplos de uso.

10. Observações

O trabalho deverá ser original. Durante a apresentação, qualquer integrante poderá ser arguido sobre qualquer parte da implementação.

11. Sintaxe de utilização

`cbuild <comando> [opções]`

A sintaxe apresentada é o formato mínimo esperado. O grupo poderá acrescentar opções para as funcionalidades avançadas, desde que a sintaxe completa seja documentada no README.

Comandos disponíveis:

- `build` Compila o projeto.
- `run` Executa o programa compilado.
- `clean` Remove arquivos temporários e o executável.
- `rebuild` Executa as operações de limpeza e compilação.
- `info` Exibe informações e estatísticas do projeto.

Exemplos:

```
./cbuild build
./cbuild run
./cbuild clean
./cbuild rebuild
./cbuild info
```

12. Estrutura esperada do projeto

Estrutura mínima esperada dos projetos que serão utilizados durante os testes.

A ferramenta deverá operar sobre projetos organizados conforme a estrutura abaixo:

```
projeto/
├── src/
│   ├── main.c
│   └── ...
├── include/
│   ├── *.h
│   └── ...
├── build/
├── logs/
└── README.md
```

A busca pelos arquivos fonte deverá ser realizada de forma recursiva a partir do diretório informado.

Os testes poderão conter diretórios e arquivos adicionais. A ferramenta deverá ser capaz de operar com múltiplos arquivos fonte e cabeçalhos e não deverá depender de um único nome de arquivo além das convenções explicitamente adotadas pelo grupo.

13. Casos mínimos de teste e demonstração

- Projeto válido com múltiplos arquivos .c e .h.
- Execução do programa após compilação.
- Limpeza dos artefatos com clean.
- Recompilação com rebuild.
- Consulta de informações com info.
- Diretório inexistente.
- Projeto sem arquivos .c.
- Erro de compilação provocado intencionalmente.
- Tentativa de run sem executável disponível.
- Demonstração da compilação incremental após modificar parte do projeto.
- Demonstração do modo verboso.
- Demonstração do modo debug.
- Demonstração de pelo menos dois níveis de otimização.
- Geração de relatório.
- Leitura de arquivo de configuração.
- Consulta de um log produzido pela ferramenta.

14. Decisões de projeto deixadas para o grupo

- Organização interna dos scripts e funções.
- Estratégia de descoberta dos arquivos.
- Estratégia de compilação incremental.
- Formato e organização dos logs.
- Formato dos relatórios.
- Formato e localização do arquivo de configuração.
- Opções adicionais de linha de comando.
- Formato das mensagens exibidas ao usuário.
- Critério para contagem de linhas de código.
- Organização dos artefatos de compilação.

As decisões acima não precisam seguir uma solução única. Soluções diferentes serão aceitas quando atenderem aos requisitos funcionais e técnicos e forem justificadas adequadamente.

15. Checklist antes da entrega

- ☐ Todos os cinco comandos obrigatórios estão implementados.
- ☐ Tratamento dos erros obrigatórios está demonstrado.

- ☐ Logs contêm todos os campos exigidos.
- ☐ info apresenta as estatísticas exigidas.
- ☐ Funcionalidades adicionais obrigatórias estão implementadas.
- ☐ README, relatório, vídeo e histórico Git estão completos.